Exam2022

June 17, 2026

Denne skabelon indeholder fire sektioner: **Opgave 1** til **Opgave 4**. Hver sektion har fire underspørgsmål: **Spørgsmål 1** til **Spørgsmål 4**. En notebook konverteres til pdf ved at køre: jupyter nbconvert –to pdf ExamTemplate2026.ipynb i terminalen. Når notebooken eksporteres/printes til PDF, starter hver ny opgave på en ny side.

0.1 Kopiérbar blok til indsættelse af billeder

0.1.1 Markdown-metode

Kopiér linjen nedenfor til en Markdown-celle og ret filnavn/størrelse efter behov:

```
<img src="images/dit_billede.png" alt="Kort beskrivelse" width="600">
```

0.1.2 Python-metode

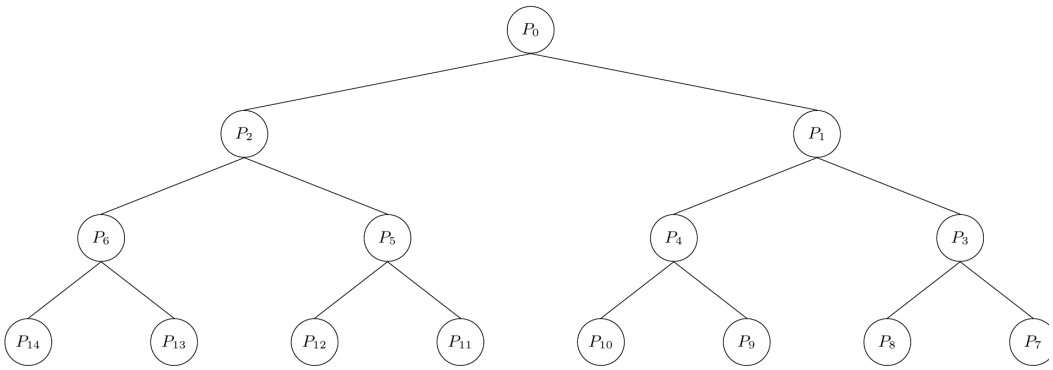
Kør kodecellen nedenfor, og brug funktionen `indsæt_billede(...)` i dine svar.

[1]:
```
from IPython.display import Image, display

def indsæt_billede(sti, bredde=600):
    """Viser et billede i notebooken.

    Parametre:
    sti: Filsti til billedet, fx 'images/figur1.png'
    bredde: Billedets bredde i pixels, fx 600
    """
    display(Image(filename=sti, width=bredde))

indsæt_billede('tree.png', bredde=600)
```



[2]:
```
# Imports
import numpy as np
import pulp as PLP
```

1 Opgave 1

1.1 Spørgsmål 1

I model this as a transport problem in the class below:

[3]:
```python
import numpy as np
import pulp as PLP

class TransportProblem:
    """Implementation follows Netværksmodeller from week 7"""
    def __init__(self, cost_matrix, supply, demands):
        # VERY LARGE NUMBER
        self.M = 10**6
        self.cost_matrix = cost_matrix
        self.demands = demands
        self.supply = supply

        self.demand_surplus = False
        self.supply_surplus = False
        self.balanced = False
        self.constraints = "NOT ADDED"

        # Check if problem is valid
        if sum(self.supply) < sum(self.demands):
            print("DEMAND SURPLUS, ADD DUMMY SUPPLIER")
            self.difference = abs(sum(self.demands) - sum(self.supply))
            self.demand_surplus = True

        elif sum(self.supply) > sum(self.demands):
            self.difference = abs(sum(self.demands) - sum(self.supply))
            self.supply_surplus = True
            print("CAPACITY SURPLUS, ADD DUMMY COSTUMER")
        else:
            print("SUPPLY MEETS DEMAND")
            self.balanced = True


        # m suppliers
        self.m = len(self.supply)
        self.supplier_range = range(self.m)
        # n costumers
        self.n = len(self.demands)
        self.demands_range = range(self.n)
        self.name = "TransportProblem"

        if not self.balanced:
            self.add_dummy()
```

```
        # Model definition
        self.model = PLP.LpProblem( name = self.name, sense = PLP.LpMinimize)
        # Decision variables
        self.x = PLP.LpVariable.dicts( name = "x", indices= (self.
↪supplier_range, self.demands_range), lowBound= 0)




    def construct_constraints(self):
        # Objective
        self.model += PLP.lpSum(self.cost_matrix[i][j] * self.x[i][j]
                                for i in self.supplier_range
                                for j in self.demands_range), "Objective"
        # We must be able to supply
        for i in self.supplier_range:
            self.model += PLP.lpSum(self.x[i][j] for j in self.demands_range)␣
↪<= self.supply[i], f"Capacities{i}"
        # We must meet demand
        for j in self.demands_range:
            self.model += PLP.lpSum(self.x[i][j] for i in self.supplier_range)␣
↪== self.demands[j], f"Demands{j}"
        # Non-negativity is enforced in variable definition
        self.constraints = "ADDED"

    def add_dummy(self):
        # Adds dummy according to surplus
        if self.demand_surplus:
            print(f"Demand surplus, adding dummy supplier with supply: {self.
↪difference}")
            self.supply.append(self.difference)
            self.m = len(self.supply)
            self.supplier_range = range(self.m)
            self.dummy_index = len(self.supply) - 1
            # Add zero cost row to matrix
            self.cost_matrix = np.vstack((self.cost_matrix, np.zeros(self.n)))

        if self.supply_surplus:
            print(f"Supply surplus, adding dummy customer with demand: {self.
↪difference}")
            self.demands.append(self.difference)
            self.n = len(self.demands)
            self.demands_range = range(self.n)
            self.dummy_index = len(self.demands) - 1
            # Add zero cost row to matrix
```

```
            self.cost_matrix = np.hstack((self.cost_matrix, np.zeros((self.m,␣
↪1))))

    def solve(self, quiet = True, postive_variables_only = True):
        if self.constraints != "ADDED":
            raise ValueError("You must add the constraint before solving")
        self.constraints = "NOT ADDED"
        # Solve quietly
        print()
        self.model.solve(PLP.PULP_CBC_CMD(msg = 0 if quiet else 1))
        # Print af loesningens status
        print("Status:", PLP.LpStatus[self.model.status])

        # Print of values of the decision variables, with option to only print␣
↪positive variables

        if postive_variables_only:
            epsilon = 1e-5
            condition = lambda v: v.varValue > epsilon
        else:
            condition = None
        if condition is not None:
            print("Solution gives the following positive variables:\n")
            for v in self.model.variables():
                if condition(v):
                    print(v.name, "=", v.varValue)
        else:
            print("Solution gives the following variables:\n")
            for v in self.model.variables():
                print(v.name, "=", v.varValue)

        # Print af den optimale objektfunktionsvaerdi
        print("Value of Objective function. = ",
              PLP.value(self.model.objective))

    def print_transport_details(self, epsilon=1e-5, one_indexed = False, msg =␣
↪True):
        """Prints the amount and cost details for all active transport routes.
↪"""
        print("\n--- Transport Route Details ---")
        if one_indexed:
            print("\n--- Transport Route Details (1-indexed) ---")
            idx = 1
        else:
            idx = 0
        self.model.solve(PLP.PULP_CBC_CMD(msg= True if msg else False))
        # Make sure the model has been solved
```

```
        if self.model.status != PLP.LpStatusOptimal:
            print("Model has not been solved to optimality yet.")
            return

        total_calculated_cost = 0

        for i in self.supplier_range:
            for j in self.demands_range:
                amount = self.x[i][j].varValue
                unit_cost = self.cost_matrix[i][j]

                route_cost = amount * unit_cost
                total_calculated_cost += route_cost
                if not self.balanced:
                    if self.supply_surplus:
                        if j != self.dummy_index:
                            print(f"Supplier {i + idx} sends {amount} units to␣
↪Customer {j + idx} "
                                  f"| Unit Cost: {unit_cost} | Route Cost:␣
↪{route_cost}")
                        else:
                            print(f"Supplier {i + idx} sends {amount} units to␣
↪Dummy Customer {j + idx} "
                                  f"| Unit Cost: {unit_cost} | Route Cost:␣
↪{route_cost}")
                    if self.demand_surplus:
                        if i != self.dummy_index:
                            print(f"Supplier {i + idx} sends {amount} units to␣
↪Customer {j + idx} "
                                  f"| Unit Cost: {unit_cost} | Route Cost:␣
↪{route_cost}")
                        else:
                            print(f"Dummy Supplier {i + idx} sends {amount}␣
↪units to Customer {j + idx} "
                                  f"| Unit Cost: {unit_cost} | Route Cost:␣
↪{route_cost}")
                else:
                    print(f"Supplier {i + idx} sends {amount} units to Customer␣
↪{j + idx} "
                          f"| Unit Cost: {unit_cost} | Route Cost:␣
↪{route_cost}")

        print("-" * 31)
        # Model objective
        print(f"Model Objective Value: {PLP.value(self.model.objective)}")
```

Using the class, we can solve the problem at hand

[4]:
```
demand = [225, 125, 150 , 325, 175]
supply = [400, 200, 300, 100]

cost = [[3, 6, 6 ,7, 7],
        [2, 9 ,4 ,9 ,4],
        [8, 9 ,3 ,1 ,7],
        [5, 4, 8, 5 ,2]]

TP = TransportProblem(cost, supply, demand)
TP.construct_constraints()
TP.solve()
```

```
SUPPLY MEETS DEMAND

Status: Optimal
Solution gives the following positive variables:

x_0_0 = 225.0
x_0_1 = 125.0
x_0_2 = 25.0
x_0_3 = 25.0
x_1_2 = 125.0
x_1_4 = 75.0
x_2_3 = 300.0
x_3_4 = 100.0
Value of Objective function. =  3050.0
```

2 Spørgsmål 2

We can use vogels aproximation, it works in the following way:

In Vogel's Approximation Method, an edge is selected iteratively and then assigned as much flow as possible.

The edge is chosen as the least-cost edge in a row or column for which the difference between the cheapest and the second-cheapest cost in that row or column is maximal.

The value (flow) assigned to the edge is the maximum possible amount, determined by the supplier's remaining capacity and the customer's remaining demand (i.e., the minimum of the two).

Since in each iteration either a remaining supply or a remaining demand is reduced to zero, there can be at most n+m−1 iterations in a balanced transportation problem.

According to the pseudocode, however, there are no further choices to make once only a single supplier or a single customer remains. This situation occurs after at most n+m−3 iterations.

[5]:
```
from modeller.vogels import vogels_table
df = vogels_table(cost, supply, demand)[0]
df
```

[5]:

	Demand 1	Demand 2	Demand 3	Demand 4	Demand 5	Supply	delta
Supply 1	3	6	6	7	7	400	3
Supply 2	2	9	4	9	4	200	2
Supply 3	8	9	3	1	7	300	2
Supply 4	5	4	8	5	2	100	2
Demand	225	125	150	325	175		
delta	1	2	1	4	2		

We see that the largest delta is on column 4, and min(300,325) is from a row, as such we remove the row, and track that we seed 300 over the edge (3,4), since supply is fully used we remove the row.

[6]:
```
demand[3] = demand[3] - 300
df = vogels_table(cost, supply, demand, active_rows=[True, True, False,␣
 ↪True])[0]
df
```

[6]:

	Demand 1	Demand 2	Demand 3	Demand 4	Demand 5	Supply	delta
Supply 1	3.0	6.0	6.0	7.0	7.0	400.0	3.0
Supply 2	2.0	9.0	4.0	9.0	4.0	200.0	2.0
Supply 3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Supply 4	5.0	4.0	8.0	5.0	2.0	100.0	2.0
Demand	225.0	125.0	150.0	25.0	175.0		
delta	1.0	2.0	2.0	2.0	2.0		

The largest delta is now at row one, as such we supply the cheapest edge (1,1) with min(225,400), since we meet demand we remove the column.

[7]:
```
supply[0] = supply[0] - 225
df = vogels_table(cost, supply, demand, active_rows=[True, True, False, True],␣
 ↪active_cols=[False] + 4*[True])[0]
df
```

[7]:

	Demand 1	Demand 2	Demand 3	Demand 4	Demand 5	Supply	delta
Supply 1	NaN	6.0	6.0	7.0	7.0	175.0	0.0
Supply 2	NaN	9.0	4.0	9.0	4.0	200.0	0.0
Supply 3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Supply 4	NaN	4.0	8.0	5.0	2.0	100.0	2.0
Demand	NaN	125.0	150.0	25.0	175.0		
delta	NaN	2.0	2.0	2.0	2.0		

2.1 Spørgsmål 3

The Northwest Corner Rule is a method used to obtain an initial feasible solution to a transportation problem.

The idea is to start with the first supplier and the first customer, which corresponds to the top-left (northwest) corner of the transportation table.

Allocate as many units as possible from the current supplier to the current customer.

If the supplier's available supply is exhausted, move down to the next supplier. If the customer's demand is completely satisfied, move to the next customer on the right.

Continue this process, always allocating as much as possible between the current supply and demand, until all supply has been distributed and all demand has been satisfied.

The method focuses only on matching supply and demand in a systematic way and does not take transportation costs into account.

Its purpose is to quickly generate a feasible starting solution that can later be improved using optimization methods.

[8]:
```
import copy
class NordVest():
    """
    This implementation of Nordvesthjørneregelen follows from Uge 8 Transport -␣
 ↪Steppingstone of Vogels Approksimation, slide 8
 """
    def __init__(self, supply, demand):
        self.supply = supply
        self.demand = demand
        if sum(self.supply) - sum(self.demand) != 0:
            raise ValueError("Sum of supply and demand must be equal")

        self.s_prime = copy.copy(self.supply)
        self.d_prime = copy.copy(self.demand)

        #Decision variable as dict
        self.n = len(self.supply)
        self.m = len(self.demand)
        self.x = {(i,j) : 0 for i in range(self.n) for j in range(self.m)}

    def solve(self, one_indexed = True, print_sol = True):
        idx = 1 if one_indexed else 0
        i = 0
        j = 0
        while i < self.n and j < self.m:
            self.x[(i,j)] = min(self.s_prime[i], self.d_prime[j])
            if i < self.m or j < self.n - 1:
                self.s_prime[i] = self.s_prime[i] - self.x[(i,j)]
                self.d_prime[j] = self.d_prime[j] - self.x[(i,j)]
                if self.d_prime[j] > 0:
                    i = i + 1
                else:
                    j = j + 1
        if print_sol:
            for key, value in self.x.items():
                if value > 0:
```

```
                    print(f"Assign {value} from supplier {key[0] + idx} to␣
↪customer {key[1] + idx}")



def tabulized_solution(supply, demand, sol, one_indexed=True):

    """
    Create a transportation tableau from a NordVest solution.

    Parameters
    ----------
    supply : list
        Supply for each source.
    demand : list
        Demand for each destination.
    sol : dict
        Solution dictionary from NordVest.x
    one_indexed : bool
        If True labels rows/columns as F1, F2,... and K1, K2,...

    Returns
    -------
    pandas.DataFrame
    """
    import pandas as pd
    n = len(supply)
    m = len(demand)

    row_names = [f"F{i+1}" for i in range(n)] if one_indexed else [f"F{i}" for␣
↪i in range(n)]
    col_names = [f"K{j+1}" for j in range(m)] if one_indexed else [f"K{j}" for␣
↪j in range(m)]

    # Build matrix from solution dictionary
    data = []
    for i in range(n):
        row = [sol.get((i, j), 0) for j in range(m)]
        row.append(supply[i])  # Supply column
        data.append(row)

    columns = col_names + ["Supply"]

    df = pd.DataFrame(data, index=row_names, columns=columns)

    # Demand row
    df.loc["Demand"] = demand + [""]
```

```
    return df
demand = [225, 125, 150 , 325, 175]
supply = [400, 200, 300, 100]

cost = [[3, 6, 6 ,7, 7],
        [2, 9 ,4 ,9 ,4],
        [8, 9 ,3 ,1 ,7],
        [5, 4, 8, 5 ,2]]
Nordvest = NordVest(supply, demand)
Nordvest.solve(print_sol=False)

print("Cost of solution: ", sum([Nordvest.x[(i,j)] * cost[i][j] for i in␣
 ↪range(len(cost)) for j in range(len(cost[0]))]))

tabulized_solution(supply, demand, Nordvest.x)
```

```
Cost of solution:  3975
```

[8]:

	K1	K2	K3	K4	K5	Supply
F1	225	125	50	0	0	400
F2	0	0	100	100	0	200
F3	0	0	0	225	75	300
F4	0	0	0	0	100	100
Demand	225	125	150	325	175	

Since we know that the optimal solution has lower cost, we can use the stepping stone method to improve the solution. We see that sending from F_2 to K_1 is cheap but unused, as such we can reallocate row capacity between rows 1 and 2.

[9]:
```
# Move 100 from F2 > K4 to F2 to K1
Nordvest.x[(1,0)] += 100
Nordvest.x[(1,3)] -= 100
# Move 100 from F1 > K1 to F1 > K4 to balance
Nordvest.x[(0,0)] -= 100
Nordvest.x[(0,3)] += 100
print("Cost of solution: ", sum([Nordvest.x[(i,j)] * cost[i][j] for i in␣
 ↪range(len(cost)) for j in range(len(cost[0]))]))

tabulized_solution(supply, demand, Nordvest.x)
```

```
Cost of solution:  3675
```

[9]:

	K1	K2	K3	K4	K5	Supply
F1	125	125	50	100	0	400
F2	100	0	100	0	0	200
F3	0	0	0	225	75	300
F4	0	0	0	0	100	100
Demand	225	125	150	325	175	

And we see an improvement in the solution

2.2 Spørgsmål 4

Indsæt din besvarelse her.

[10]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

3 Opgave 2

Skriv din besvarelse nedenfor.

3.1 Spørgsmål 1

Indsæt din besvarelse her.

[11]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

3.2 Spørgsmål 2

Indsæt din besvarelse her.

[12]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

3.3 Spørgsmål 3

Indsæt din besvarelse her.

[13]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

3.4 Spørgsmål 4

Indsæt din besvarelse her.

[14]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

4 Opgave 3

Skriv din besvarelse nedenfor.

4.1 Spørgsmål 1

Indsæt din besvarelse her.

[15]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

4.2 Spørgsmål 2

Indsæt din besvarelse her.

[16]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

4.3 Spørgsmål 3

Indsæt din besvarelse her.

[ ]:
```
%%sql
```

[17]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

4.4 Spørgsmål 4

Indsæt din besvarelse her.

[18]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

5 Opgave 4

Skriv din besvarelse nedenfor.

5.1 Spørgsmål 1

Indsæt din besvarelse her.

[19]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

5.2 Spørgsmål 2

Indsæt din besvarelse her.

[20]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

5.3 Spørgsmål 3

Indsæt din besvarelse her.

[21]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```

5.4 Spørgsmål 4

Indsæt din besvarelse her.

[22]:
```
# Beregninger/kode til dette spørgsmål kan skrives her
```